
DAWN-SRW: State-Conditioned Programs for Learned State Transitions

Seungho Choi¹

Abstract

Dense language models operate over learned high-dimensional states, but the operations producing their transitions and the conditions governing their use remain largely implicit. We ask whether learned transition computation can be organized around explicit computational objects whose conditions of use and contributions to state transition can be identified, measured, and intervened on. We distinguish two computational roles and make them explicit through one control–operation factorization: $C(x)$ represents when, whether, and to what degree a learned operation contributes, while $O_i(x)$ represents the transformation it performs. The current state induces this applicability profile; composition combines the resulting controlled operation contributions into a state transition. This motivates a *State-Conditioned Program*: a computational formulation of learned conditional operation application. The narrower architectural hypothesis is that transition-producing computation can be organized into identifiable learned operations under explicit state-conditioned control over their contributions. DAWN-SRW provides one concrete realization, using rank-1 read-write (RW) operators, an operation-address space, normalized additive composition, full candidate scope, and dense RW execution. At 393.8M parameters and 39.999B tokens, DAWN-SRW supports viability relative to the matched dense control: C4 cross-entropy is 2.9402 versus 2.9359 and the six-task zero-shot mean is 0.4629 versus 0.4597. State-conditioned exact applicability support provides evidence of conditional organization. A discovery-frozen 4,096-site Q/K-centered operator circuit passes suppression, four matched-control families, and exact restoration on validation and held-out indirect-object-identification (IOI) inputs, establishing causal accessibility; fail-closed results delimit unsuccessful localization. They do not establish formal operational or

human-readable semantics, benchmark superiority, hardware efficiency, or an intrinsic or unique decomposition.

1. Introduction

Language models repeatedly transform learned high-dimensional representations. If X_ℓ denotes the sequence representation at layer ℓ , a block implements, at a coarse level,

$$X_\ell \longrightarrow X_{\ell+1}.$$

These transformations support powerful computation, but their internal computational organization remains largely implicit. This does not mean that dense Transformers lack state dependence or conditional structure. Rather, their dense transformations do not natively expose a stable architectural interface that separates the operations available to a state, the conditions governing their use, and their individual contributions to the transition.

We therefore ask:

Can learned transition computation be organized around explicit computational objects whose conditions of use and contributions to state transition can be identified, measured, and intervened on?

To make this question architecturally testable, we distinguish two computational roles: what transformation is performed, and when, whether, and to what degree it contributes to the current state transition.

We make these roles explicit through one control–operation factorization:

$$\underbrace{C(x)}_{\text{when / whether / how much}} \quad \text{and} \quad \underbrace{\{O_i(x)\}}_{\text{what transformation}}.$$

$C(x)$ represents when, whether, and to what degree a learned operation contributes, while $O_i(x)$ represents the transformation it performs. Language-model computation proceeds through evolving learned representation states. At each transition-producing interface, the current representation is the immediate computational state. We therefore make control explicitly state-conditioned: the current state

¹Independent Researcher. Correspondence to: Seungho Choi <madst0614@gmail.com>.

induces an applicability profile $C(x)$ over learned operations. State-conditioned applicability modulates each operation response, producing controlled operation contributions. An explicit composition rule specifies how those contributions form the resulting state transition.

Together, we organize this computation as:

current state $\xrightarrow{\text{state-conditioned applicability and learned operations}}$
 $\rightarrow$ controlled operation contributions
 $\rightarrow$ composition $\rightarrow$ next state.

This organization generalizes a familiar feature of program execution: operations are applied conditionally to a current state. Here both the conditions of applicability and the operations are learned, applicability is continuous-valued rather than restricted to a discrete predicate, and multiple controlled operation contributions may be composed into one transition. Operational accounts such as Plotkin’s provide limited conceptual lineage for describing computation through explicit transitions between configurations (Plotkin, 1981); they do not supply the control–operation factorization, a formal semantics for DAWN-SRW, or a claim of formal equivalence.

We call this computational formulation a *State-Conditioned Program*: an explicit organization in which a current learned state induces state-conditioned applicability over learned operations, controlled operation contributions are composed, and a resulting state transition is formed,

$$x \mapsto (C(x), \{O_i(x)\}) \mapsto F(x) \rightarrow x'.$$

Here x is the current representation state, $C(x)$ is explicit state-conditioned control, $c_i(x)$ is the applicability of learned operation O_i , $c_i(x)O_i(x)$ is its controlled operation contribution, and $F(x)$ is the composition of those contributions into a transition-producing result. This use of *program* requires no symbolic program text, discrete instructions, human-readable predicates, or human-readable operator semantics, and it does not define a formal structural operational semantics.

To constructively test this formulation in language modeling, we study the narrower architectural hypothesis that transition-producing computation can be organized into identifiable learned operations under explicit state-conditioned control over their contributions. DAWN-SRW is one concrete realization of this architectural hypothesis. Here, identifiable means referable as a stable architectural object whose contribution can be measured or intervened on; it does not imply semantic purity, uniqueness, or an intrinsic decomposition. The factorization makes applicability and individual operation contributions explicit measurement and intervention interfaces.

The state-conditioned control design problem is to specify how a high-dimensional state determines operation applica-

bility. At the abstract level, this is an explicit applicability profile,

$$x \mapsto C_a(x) = (c_{a,1}(x), \dots, c_{a,N_a}(x)). \quad (1)$$

Here, $c_{a,i}(x) = 0$ gives exact non-participation, while positive values provide continuous-valued applicability. More generally, applicability defines a learned relation between the current state and each learned operation,

$$(x, O_{a,i}) \mapsto c_{a,i}(x).$$

This condition-like role is realized in the operation-address space: a state-derived query and operator addresses lie on the unit sphere, and cosine similarity measures angular proximity. A learned threshold bounds the query’s state-dependent angular neighborhood; addresses inside define the exact applicability support, and continuous-valued applicability determines each operation’s degree of contribution. Learning jointly organizes persistent operator addresses as type-specific spherical arrangements shared across states and depth, with state-derived queries inducing local angular neighborhoods. These contributions form the state transition; the exact construction is given in Section 3.5.

On the operation side, DAWN-SRW uses a rank-1 read-write (RW) realization and composes its controlled contributions as

$$O_i(x) = \langle x, \bar{r}_i \rangle \bar{w}_i, \quad (2)$$

$$F(x) = \frac{\sqrt{D/L}}{Z(x)} \sum_i c_i(x) O_i(x), \quad c_i(x) \in [0, 1]. \quad (3)$$

The read response $\langle x, \bar{r}_i \rangle$ provides a signed state-dependent scalar and $\bar{w}_i$ its output direction, making each RW operator a directional transition component. Each product $c_i(x)O_i(x)$ is a controlled operation contribution. DAWN-SRW chooses normalized additive composition to combine multiple such contributions into one transition-producing output. At a fixed state, this preserves an explicit operator-wise additive decomposition; because applicability depends on the state, the overall map $x \mapsto F(x)$ is generally nonlinear.

DAWN-SRW additionally reuses persistent type-specific operator pools across depth and evaluates applicability over the full type-specific operator pool. These identity and candidate scope choices, together with dense RW execution, are implementation decisions rather than requirements of the formulation.

The empirical study tests this concrete realization under the evaluated conditions through three ordered claims: viability, conditional organization, and causal accessibility. Fail-closed results delimit where the mechanistic localization procedure does not succeed; efficiency is not a headline claim.

Our contributions are:

1. **A State-Conditioned Program formulation for transition-producing language-model computation and one architectural realization.** We formulate transition-producing language-model computation as a State-Conditioned Program. We then study the narrower architectural hypothesis that transition-producing computation can be organized into identifiable learned operations under explicit state-conditioned control over their contributions. DAWN-SRW provides one concrete realization through persistent rank-1 RW operators, an operation-address space, normalized additive composition across Q/K/V and the residual-state transition (RST), full candidate scope, and dense RW execution.
2. **A constructive empirical test with bounded evidence.** We test viability at 400M scale, report descriptive results for the 1.3B DAWN-SRW checkpoint (1.272B parameters), provide evidence of conditional organization through state-conditioned exact applicability support, and test causal accessibility through a discovery-frozen IOI circuit confirmed by suppression, matched controls, and exact restoration. Fail-closed results bound where the same localization procedure did not succeed.

2. Related Work

2.1. State-dependent and dynamically parameterized transformations

State-dependent parameterizations have long precedents in nonlinear control and neural sequence models. State-dependent Riccati techniques use state-dependent coefficient representations (Cloutier, 1997), cited here as conceptual lineage rather than mathematical equivalence. Multiplicative RNNs use input-dependent transition matrices (Sutskever et al., 2011), input-switched affine networks use input-dependent recurrent weights (Foerster et al., 2017), and HyperNetworks generate weights for another network (Ha et al., 2017). Together these works establish state- or input-dependent transformation as precedent. DAWN-SRW studies the narrower organization of transition-producing language-model computation into learned operations and explicit state-conditioned control over their contributions.

2.2. Conditional and modular computation

Sparsely-gated mixtures of experts route among expert networks (Shazeer et al., 2017), Switch Transformers scale simple routing (Fedus et al., 2022), and ModuleFormer routes tokens over attention and feed-forward modules (Shen et al., 2023). PEER applies product-key retrieval to million-scale

tiny experts (He, 2024), while OmniMoE uses vector-level Atomic Experts and structured routing (Shi et al., 2026).

Cross-depth sharing also has direct precedents: Universal Transformers recur over shared layers (Dehghani et al., 2019), MoEUT shares expertized feed-forward and attention computation (Csordás et al., 2024a), MoUE reuses a layer-agnostic expert pool (Chen et al., 2026), and UniPool exposes a global pool through per-layer routers (Huang et al., 2026).

Beyond feed-forward routing, SwitchHead routes attention-side experts (Csordás et al., 2024b), UMoE shares experts across attention and FFN computation (Yang et al., 2025), and Compositional Attention separates query–key search from value retrieval (Mittal et al., 2022). These lines establish state-dependent routing, fine-grained expert computation, cross-depth reuse, and factorized attention as strong precedents. DAWN-SRW instead studies type-specific RW operators under state-conditioned control over their contributions across Q/K/V/RST computation.

2.3. Read-write operators and memory views

Geva et al. interpret Transformer feed-forward layers as key-value memories with input-sensitive coefficients over output directions (Geva et al., 2021), and later decompose their additive updates into parameter-vector sub-updates traced through vocabulary-space predictions (Geva et al., 2022). DAWN-SRW instead makes $c_i(x) \underbrace{\langle x, \bar{r}_i \rangle \bar{w}_i}_{O_i(x)}$ an architectural

operation contribution under state-conditioned applicability. Product Key Memory provides lineage for factorized keys and structured retrieval (Lample et al., 2019), scalable memory layers extend content-addressable trainable memories (Berges et al., 2024), Fast Weight Programmers provide an outer-product programming precedent (Schlag et al., 2021), and MoRAM uses rank-1 associative-memory experts with intrinsic keys (Lu et al., 2026). These are precedents for memory, retrieval, outer-product structure, and rank-1 experts; DAWN-SRW studies RW operators under a separate state-conditioned applicability profile.

2.4. Learned rules and dynamically composed functions

Neural Production Systems instantiate learned condition–action production rules: state- and entity-dependent matching selects rule–entity bindings, learned rules update entity state, and the studies emphasize object-centric structured dynamics and systematic generalization (Didolkar et al., 2021). Neural Interpreters factorize self-attention inference into reusable functions represented by signatures and codes. Learned input types and signatures occupy a hyperspherical type space whose matching controls function access; a shared code-conditioned interpreter executes and

dynamically composes function streams across width and depth (Rahaman et al., 2021). Neural Attentive Circuits separate a circuit generator from its executor: signatures determine module connectivity, codes condition module computation, and the executor runs the generated configuration (Weiss et al., 2022). These works provide concrete precedents for learned conditions of computation: state- or entity-dependent matching, type–signature compatibility, and generated modular configurations determine which learned rules or functions become active.

DAWN-SRW addresses the same broad problem of learned conditional access, but gives it the explicit role of state–operation applicability, $(x, O_i) \mapsto c_i(x)$, over learned operations. Neural Interpreters and Neural Attentive Circuits use signatures and codes to address or configure computation realized through shared interpreters or modules. DAWN-SRW instead uses persistent RW operators, $O_i(x) = \langle x, \bar{r}_i \rangle \bar{w}_i$, and derives each operator address from that transformation’s read/write parameters. It applies this organization to transition-producing Q/K/V/RST computation and operator-level measurement and intervention in an autoregressively pretrained language model.

2.5. Neural programming and programming interpretations

Neural Programmer-Interpreters execute programs from supervised traces (Reed & de Freitas, 2016), Neural Programmer differentially selects operations and data (Neelakantan et al., 2016), and Fast Weight Programmers connect outer-product computation to instructions (Schlag et al., 2021). RASP gives a programming-language-level model of Transformer computation and realizability (Weiss et al., 2021). Thus neural programs, operation selection, and Transformer computation as program all have precedent. DAWN-SRW instead studies learned operations under state-conditioned control as native transition-producing architecture, without symbolic source code or a claim of formal semantic equivalence.

2.6. Mechanistic analysis and causal intervention

Mechanistic interpretability studies localized components through causal intervention. IOI work isolates an attention-head circuit (Wang et al., 2023), AtP* approximates activation patching at scale (Kramár et al., 2024), and RAVEL evaluates localization against multiple causal criteria (Huang et al., 2024). This literature distinguishes candidate localization from causal confirmation. DAWN-SRW supplies RW-operator intervention targets, while its discovery freeze, suppression, matched controls, and exact restoration test the specified IOI circuit without implying general semantic interpretability.

Together, these lines establish precedents for state depen-

dence, conditional routing, learned functions, modular language-model computation, geometric access, read/write decomposition, neural programs, and causal intervention. DAWN-SRW organizes these established roles within a State-Conditioned Program formulation and tests one realization: RW operators under state-conditioned control over their contributions, evaluated for viability, conditional organization, and causal accessibility.

3. DAWN-SRW

3.1. State Transitions and Control–Operation Factorization

Let

$$X_\ell = (x_{\ell,1}, \dots, x_{\ell,T}) \in \mathbb{R}^{T \times D}$$

denote the sequence representation at depth ℓ . A Transformer-like block produces state transitions through attention and a subsequent residual transformation:

$$Y_\ell = X_\ell + \text{Attn}_\ell(X_\ell), \quad (4)$$

$$X_{\ell+1} = Y_\ell + R_\ell(Y_\ell). \quad (5)$$

The architectural hypothesis formalizes one way of organizing transition-producing computation: identifiable learned operations are placed under explicit state-conditioned control over their contributions. DAWN-SRW realizes this hypothesis through reusable RW operators, an applicability profile, and normalized additive composition. For a token state $x \in \mathbb{R}^D$ and computation type $a \in \{Q, K, V, \text{RST}\}$, define

$$F_a : \mathbb{R}^D \rightarrow \mathbb{R}^D. \quad (6)$$

Here F_Q, F_K, F_V parameterize relational interaction and F_{RST} parameterizes a direct residual-state update. Rank-1 RW operators, the operation-address space, persistence, and normalized additive composition are realization choices, not requirements of the architectural hypothesis. We use *operation* for the abstract transformation, *operator* for an indexed learned architectural object implementing an operation, and *RW operator* for the rank-1 realization used here.

3.2. Read-Write Operators

Each RW operator owns learned read and write parameters. With $\text{unit}(z) = z / \max(\|z\|_2, \epsilon)$, define normalized directions $\bar{r}_{a,i} = \text{unit}(r_{a,i})$ and $\bar{w}_{a,i} = \text{unit}(w_{a,i})$. The RW operator executes

$$O_{a,i}(x) = \langle x, \bar{r}_{a,i} \rangle \bar{w}_{a,i} = \bar{w}_{a,i} \bar{r}_{a,i}^\top x. \quad (7)$$

The read direction produces a state-dependent scalar response and the write direction its output direction, making this a rank-1 input-dependent transformation. Q/K/V RW operators compose attention interfaces; RST RW operators compose a direct residual-state update.

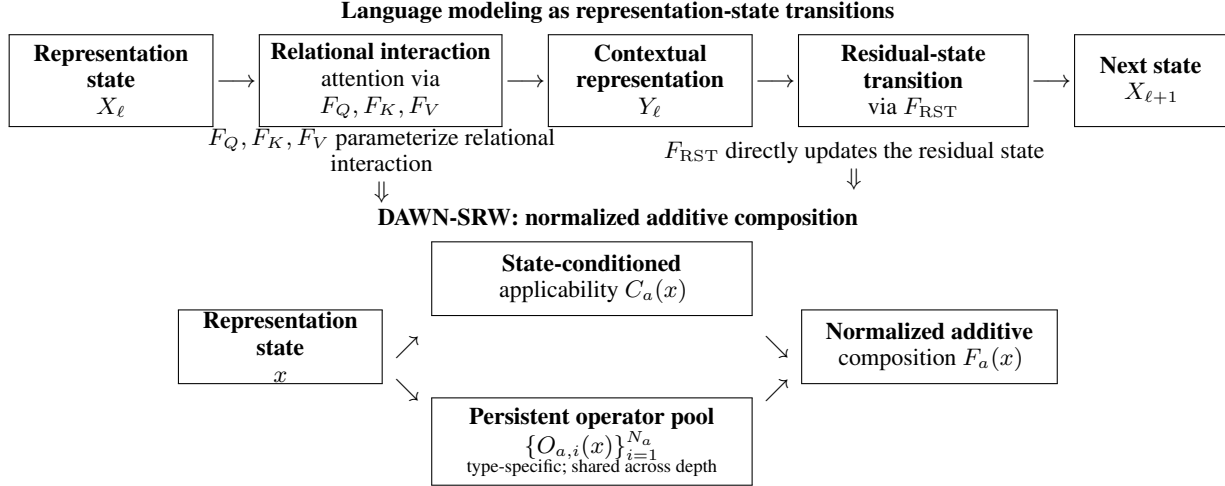


Figure 1. The two-level construction. Top: a language-model block produces state transitions through relational interaction and a subsequent residual-state transition. Bottom: x separately determines the applicability profile and state-dependent responses of a persistent, type-specific operator pool shared across depth. Each product $c_{a,i}(x)O_{a,i}(x)$ is a controlled operation contribution; normalized additive composition combines these contributions into the transition-producing output $F_a(x)$. Here, $q_a(x), k_{a,i}, s_{a,i}(x), \tau_a(x)$ implement applicability in the operation-address space; RW operators remain in the full D -dimensional representation space, and physical RW execution is dense.

3.3. State-Conditioned Applicability

For each computation type, the current state induces an applicability profile over its N_a reusable RW operators,

$$C_a(x) = (c_{a,1}(x), \dots, c_{a,N_a}(x)), \quad (8)$$

whose coordinates determine whether and to what degree each RW operator participates. Section 3.5 defines the state-derived query $q_a(x)$ and operator address $k_{a,i}$. Their applicability score and applicability threshold are

$$s_{a,i}(x) = q_a(x)^\top k_{a,i}, \quad (9)$$

$$\tau_a(x) = -1 + 2\sigma(xu_a + \beta_a). \quad (10)$$

The evaluated `linear_angular` rule defines

$$c_{a,i}(x) = \text{clip}\left(\frac{s_{a,i}(x) - \tau_a(x)}{\max(1 - \tau_a(x), 10^{-4})}, 0, 1\right). \quad (11)$$

We call $c_{a,i}(x) \in [0, 1]$ the *continuous-valued applicability* of operator i at state x . Its exact applicability support is

$$\begin{aligned} \mathcal{I}_a(x) &= \{i : c_{a,i}(x) > 0\} \\ &= \{i : s_{a,i}(x) > \tau_a(x)\}. \end{aligned} \quad (12)$$

When $s_{a,i}(x) \leq \tau_a(x)$, an operator contributes exactly zero; when $s_{a,i}(x) > \tau_a(x)$, $0 < c_{a,i}(x) \leq 1$ varies continuously with the applicability score. Thus the same applicability score and applicability threshold specify exact non-participation and continuous-valued applicability.

3.4. Normalized Additive Composition

DAWN-SRW uses normalized additive composition to combine controlled RW-operator contributions into the output

$F_a(x)$. With normalization factor $Z_a(x)$,

$$Z_a(x) = \left[\max\left(\sum_i c_{a,i}(x), 1\right) \right]^{p_a}, \quad (13)$$

$$F_a(x) = \frac{\sqrt{D/L}}{Z_a(x)} \sum_i c_{a,i}(x) \bar{w}_{a,i} \bar{r}_{a,i}^\top x. \quad (14)$$

In each term, $c_{a,i}(x)$ specifies state-conditioned applicability and $\bar{w}_{a,i} \bar{r}_{a,i}^\top x$ is the operation response; their product is a controlled operation contribution. This additive form is a deliberate realization choice: at a fixed state, it preserves an explicit operator-wise decomposition of the transition-producing output, making individual controlled contributions directly measurable and available as intervention targets; the reported interventions suppress or restore selected numerator terms while leaving the normalization factor unchanged. A nonlinear mixer would generally introduce cross-operation interactions that obscure this exact decomposition; because both $c_{a,i}(x)$ and $Z_a(x)$ are state-dependent, the overall mapping $x \mapsto F_a(x)$ remains generally nonlinear. Exact composition parameters are listed in Appendix A.

3.5. Operation-Address Space

In this realization, evaluating the condition under which an operation contributes becomes a learned geometric relation between the current state and that operator. The operation-address space provides one geometric realization of state–operation applicability. Representation and RW computation remain in $\mathbb{R}^D$; the query and operator

addresses $q_a(x), k_{a,i} \in \mathbb{R}^{d_c}$ are control variables. Here, $d_c = 256$ and

$$q_a(x) = \text{unit}(xP_a + b_a) \in \mathbb{R}^{d_c}. \quad (15)$$

A shared probe pair $P_r, P_w \in \mathbb{R}^{D \times d_c}$ maps functional read and write directions into the operator address

$$k_{a,i} = \text{unit}(\text{unit}(\bar{r}_{a,i}^\top P_r) \odot \text{unit}(\bar{w}_{a,i}^\top P_w)) \in \mathbb{R}^{d_c}. \quad (16)$$

Each operator address is a learned function of what its RW operator reads and where it writes, rather than an independently parameterized address.

Because both $q_a(x)$ and $k_{a,i}$ are unit-normalized, they lie on the unit sphere $\mathbb{S}^{d_c-1}$. Writing $\theta_{a,i}(x)$ for the angle between them, $s_{a,i}(x) = q_a(x)^\top k_{a,i} = \cos(\theta_{a,i}(x))$, so $s_{a,i}(x) > \tau_a(x)$ is equivalent to $\theta_{a,i}(x) < \arccos(\tau_a(x))$. Thus the exact applicability support corresponds geometrically to a state-dependent spherical cap, or angular neighborhood, centered at $q_a(x)$ with angular radius $\arccos(\tau_a(x))$. Within this neighborhood, $c_{a,i}(x)$ gives continuous-valued applicability that varies with the cosine applicability score above the threshold and determines to what degree the operator contributes. This is the spherical realization of $(x, O_{a,i}) \mapsto c_{a,i}(x)$; exact implementation details are re-stated in Appendix A.

3.6. Attention-Mediated and Residual Transitions

For $\tilde{X}_\ell = \text{LN}_\ell^{(1)}(X_\ell)$, three normalized additive compositions produce the attention interfaces:

$$\begin{aligned} Q_\ell &= F_Q(\tilde{X}_\ell), \\ K_\ell &= F_K(\tilde{X}_\ell), \\ V_\ell &= F_V(\tilde{X}_\ell), \\ A_\ell &= \text{softmax}\left(\frac{Q_\ell K_\ell^\top}{\sqrt{d_h}} + M_{\text{causal}}\right), \end{aligned}$$

$$\Delta X_{\text{attn},\ell} = A_\ell V_\ell W_\ell^O, \quad (17)$$

$$Y_\ell = X_\ell + \Delta X_{\text{attn},\ell}. \quad (18)$$

Q and K use a shared type-specific operator pool, while V uses a separate operator pool. A second normalization $\tilde{Y}_\ell = \text{LN}_\ell^{(2)}(Y_\ell)$ then queries the RST operator pool:

$$X_{\ell+1} = Y_\ell + F_{\text{RST}}(\tilde{Y}_\ell). \quad (19)$$

Thus RST applicability is evaluated from the contextualized post-attention state, and its normalized additive composition produces the direct residual-state update.

3.7. Cross-Depth Operator Reuse and Shared Control

Type-specific operator pools and their state-conditioned control parameterizations are shared across all L layers. Evolving normalized states therefore query the same operator

pools rather than a separately parameterized profile at each depth. The two LayerNorms and attention output projection W_ℓ^O remain layer-specific; Appendix B gives the complete sharing inventory.

3.8. Candidate Scope and Physical Execution

For computation type a , the candidate scope is the full type-specific operator pool. Thus

$$\mathcal{I}_a(x) \subseteq \{1, \dots, N_a\},$$

with all N_a operators considered as candidates before applicability is determined. Applicability support is generally a strict subset, while physical RW execution is dense. Sharding, chunking, and accounting details appear in Appendix F.

4. Experimental Setup

The primary model is DAWN-SRW 400M (393.8M parameters, approximately 40B packed C4 tokens), compared with a parameter- and training-token-matched dense Transformer control (393.9M parameters). The 1.3B checkpoint, trained on approximately 20B tokens, is descriptive only. Complete architectural and training settings are in Appendix B.

Performance evaluation uses packed C4 validation and six zero-shot tasks: LAMBADA, HellaSwag, PIQA, ARC-Easy, ARC-Challenge, and WinoGrande. The two 400M models use identical evaluation protocols; complete metrics, splits, and software details are in Appendix C.

For causal analysis, IOI discovery ranks operator sites and freezes a circuit before independent validation and held-out test. Suppression removes the frozen sites' execution numerator, four matched-control families test specificity, and exact restoration reinserts the selected same-example numerator under the unchanged normalization factor. Appendix D gives the selection criteria, unit counts, statistics, and complete intervention protocol.

5. Results

5.1. Language-model performance

Table 1 consolidates the headline results. At 400M scale, DAWN-SRW differs from the matched dense control by +0.004318918 C4 cross-entropy and +0.003220495 six-task mean. These small measured gaps establish viability through performance retention, not superiority or formal equivalence.

The descriptive 1.3B checkpoint reaches 2.835879 C4 cross-entropy and a 0.485844 six-task mean; without a matched dense control, this is not a scaling comparison. Appendix C reports C4 accuracy and perplexity, the complete task-level breakdown, and standard errors.

Table 1. Headline language-model results. The 1.3B row is descriptive.

Model	Parameters	C4 CE	Six-task mean
DAWN-SRW 400M	393.8M	2.940213	0.462872
Dense 400M	393.9M	2.935894	0.459652
DAWN-SRW 1.3B	1.272B	2.835879	0.485844

Table 2. Exact applicability support according to $c_{a,i}(x) > 0$. Q/K union counts an operator in the applicability support for Q or K at the same layer and token.

Computation	Pool	Mean support	Pool fraction
Q	4,600	487.896	10.6064%
K	4,600	386.722	8.4070%
Q/K union	4,600	812.388	17.6606%
V	14,500	1,649.994	11.3793%
RST	29,300	2,017.703	6.8864%

5.2. State-Conditioned Exact Applicability Support

At all 9,994,240 packed-C4 positions, Q, K, V, and RST each have strict-subset applicability support according to $c_{a,i}(x) > 0$ (Table 2). This provides evidence of conditional organization through state-conditioned exact applicability support under Equation 11, with strict-subset rather than full-pool participation.

Restricting RW execution to exact applicability support would substantially reduce static accounting, but the resulting estimate remains above the matched dense control; this is not measured latency, energy, or hardware speedup. Detailed accounting and Figure 2 are in Appendix F.

5.3. Frozen IOI circuit and causal intervention

Discovery and confirmation are separated: a 4,096-site Q/K-centered circuit is frozen before validation, and held-out test reuses the unchanged specification. Complete discovery statistics are in Appendix D.

Suppression causes large behavior changes on both splits and exceeds every replicate in all four matched-control families. Activation-matched controls are strongest; frozen-minus-control margin separation is 2.464297 on validation and 1.937754 on test (both $p = 0.00049975$). Exact restoration recovers more than 99.8% of the suppression effect. Together these interventions establish causal accessibility for the evaluated IOI behavior; they do not imply a unique or human-readable algorithm.

5.4. Fail-Closed Results

Fail-closed results on RAVEL, ARC, and 1.3B IOI arise from distinct preregistered criteria: localization instability, failed causal suppression, and insufficient cumulative importance, respectively. Exact statistics and decisions are in Appendix E and Table 9.

6. Discussion

6.1. What the results establish

The constructive test supports three ordered claims. First, the matched 400M comparison establishes viability under the measured protocols. Second, state-conditioned exact applicability support provides evidence of conditional organization in the evaluated model; candidate scope is the full type-specific operator pool and physical RW execution is dense. Third, the discovery-frozen IOI circuit establishes causal accessibility: suppression changes behavior, matched controls do not reproduce the effect, and exact same-example restoration recovers it on independent validation and held-out data. Fail-closed results on RAVEL, ARC, and 1.3B IOI delimit where this localization procedure did not succeed.

These findings support one explicit realization under the evaluated conditions, not an intrinsic or unique decomposition. Their practical consequence is an interface in which learned operations are referable, their state-conditioned contributions measurable, and selected contributions available for intervention.

6.2. From the Computational Question to an Open Design Space

The State-Conditioned Program formulation opens four design axes: control and applicability, operation and identity, coupling and composition, and candidate scope and physical execution. DAWN-SRW supplies one tested point—rank-1 RW operators, an operation-address space, cross-depth persistence, full candidate scope, and normalized additive composition—rather than requirements for the formulation.

Control and applicability. How should state–operation applicability be represented? DAWN-SRW derives $C_a(x)$ in a learned $d_c = 256$ operation-address space. Alternatives could use functional parameters, intrinsic read activation, separate control representations, or no geometry. Open questions concern intrinsic control dimension, locality, and whether geometric similarity corresponds to functional similarity; the present evidence establishes neither that 256 is intrinsic nor that one organization is preferable.

Operation and identity. What should constitute a reusable learned operation? Rank-1 RW operators are one granularity; alternatives could be finer, coarser, structured, or hierarchical. DAWN-SRW persists type-specific identities across depth, while other scopes could be layer-local or recurrent. Persistence makes stability measurable but does not guarantee a single function or semantic purity; operations may remain distributed or polysemantic.

Table 3. Frozen IOI circuit results. Accuracy arrows show intact to suppressed. Recovery is the fraction of the suppression effect recovered by exact same-example numerator restoration.

Split	Units	Intact margin	Suppressed margin	Drop (95% CI)	Accuracy	Recovery / restored acc.
Validation	123	3.465456	-4.399331	7.864787 [7.490080, 8.241263]	1.000000 → 0.008130	0.999661 / 0.991870
Held-out test	114	3.254720	-4.155613	7.410332 [7.016485, 7.818472]	1.000000 → 0.008772	0.998406 / 1.000000

Coupling and composition. After control and operations are made explicit, how should their controlled contributions combine into a transition? In ordinary single-branch discrete conditional execution, recomposition is effectively trivial; when state-conditioned applicability is continuous-valued, multiple operations may contribute simultaneously, making composition an explicit design problem. DAWN-SRW derives addresses from read/write parameters without identifying control variables with RW operations, multiplies each operation response by its state-conditioned applicability, and then uses normalized additive composition, partly because the additive form preserves explicit operator-wise contributions. Alternative nonlinear composition rules may trade this separability for different cross-operation interaction structure; other designs could also couple the roles differently or compose contributions sequentially, hierarchically, recurrently, or otherwise.

Candidate scope and physical execution. How much of an operator pool should candidate scope include? DAWN-SRW uses the full type-specific operator pool; restricted or state-dependent candidate scope remains an open empirical question.

6.3. From Applicability Support to Physical Execution

For a given representation state, many operators have exactly zero applicability, but this does not establish restricted candidate scope or sparse physical execution. In DAWN-SRW,

$$\mathcal{I}_a(x) \subseteq \{1, \dots, N_a\},$$

so the candidate scope is the full type-specific operator pool and physical RW execution is dense. Applicability support specifies which operators contribute; candidate scope specifies which operators control considers; physical execution specifies how computation maps to hardware. **Sparse applicability support does not by itself imply hardware-exploitable sparse computation.** The open problem is to restrict candidate scope and physical execution while preserving language-model quality and conditional organization.

6.4. Limitations

Global candidate evaluation and dense RW execution remain expensive, and performance may depend partly on the capacity of the full operator pool. Even static accounting

with RW execution restricted to exact applicability support remains 2.1168 times the matched dense control’s backbone. No production latency, energy, or hardware advantage is measured.

The study does not establish that the State-Conditioned Program organization or its control–operation factorization is intrinsic, unique, necessary, privileged, or optimal. Nor are the spherical geometry, $d_c = 256$ control dimension, or correspondence between geometric and functional organization shown to be intrinsic; the geometry may instead be a high-capacity substrate supporting broadly distributed operations.

Causal evidence is confined to the specified IOI setting. Fail-closed results on RAVEL, ARC, and 1.3B IOI delimit mechanistic generalization; the differently budgeted 400M and 1.3B checkpoints do not define a controlled scaling curve; and individual operators need not map one-to-one to human-readable concepts.

7. Conclusion

Learned state-transition computation remains organizationally implicit. The State-Conditioned Program formulation organizes learned state transitions through explicit state-conditioned control over learned operations and explicit composition of their controlled contributions; DAWN-SRW tests the narrower architectural hypothesis in language modeling through its rank-1 RW realization.

The evidence is bounded: measured 400M viability, descriptive 1.3B results, evidence of conditional organization from state-conditioned exact applicability support, and causal accessibility through a discovery-frozen IOI circuit tested by suppression, matched controls, and exact restoration. Fail-closed results mark where localization did not succeed.

The main interpretation is that one learned high-dimensional state-transition system can be organized through explicit state-conditioned control and learned operations while retaining viability, not that this decomposition is intrinsic or unique. The resulting open question is not merely whether learned computation can be factorized into state-conditioned control and learned operations, but when such a factorization becomes a useful computational decomposition—and how far it can replace or expose otherwise implicit neural computation.

A. Exact Operation-Address and Composition Equations

This appendix restates the production computation as a compact reference. Raw read and write parameters are normalized in float32 before being cast for dense contractions:

$$\bar{r}_i = \frac{r_i}{\max(\|r_i\|_2, \epsilon)}, \quad \bar{w}_i = \frac{w_i}{\max(\|w_i\|_2, \epsilon)}.$$

One pair of model-level probes is shared across QK, V, and RST. In canonical notation, the operator address is

$$k_{a,i} = \text{unit}(\text{unit}(\bar{r}_{a,i}^\top P_r) \odot \text{unit}(\bar{w}_{a,i}^\top P_w)).$$

The query and applicability threshold use separate direct projections. The `DirectTau` projection produces the implementation variable `raw_tau` = $xu_a + \beta_a$, whose transformed value is the canonical applicability threshold $\tau_a(x)$:

$$q_a(x) = \text{unit}(xP_a + b_a), \\ \tau_a(x) = -1 + 2\sigma(xu_a + \beta_a), \quad s_{a,i}(x) = q_a(x)^\top k_{a,i}.$$

For the `linear_angular` applicability rule:

$$c_{a,i}(x) = \text{clip}\left(\frac{s_{a,i}(x) - \tau_a(x)}{\max(1 - \tau_a(x), 10^{-4})}, 0, 1\right)$$

and padding rows are masked to zero. This is the continuous-valued applicability used in normalized additive composition. With normalization factor $Z_a(x) = \max(\sum_i c_{a,i}(x), 1)^{p_a}$, the production composed output before causal attention or residual addition is

$$F_a(x) = \sqrt{\frac{D}{L}} \frac{\sum_i c_{a,i}(x) \langle x, \bar{r}_{a,i} \rangle \bar{w}_{a,i}}{Z_a(x)},$$

with $p_Q = p_K = 0.5$, $p_V = 1.0$, and $p_{\text{RST}} = 1.2$. Operator rows are sharded over the model axis; local numerators and local sums of applicability values are reduced across shards to produce the global numerator and normalization factor. Chunking changes the contraction schedule, not the equation.

B. Complete Model and Training Settings

The 400M DAWN-SRW checkpoint is the primary performance and mechanistic model. The dense 400M checkpoint is only a matched performance control. The 1.3B DAWN-SRW checkpoint is used for descriptive performance and one IOI localization replication that produced a fail-closed result. The still-incomplete dense 1.3B run is excluded.

The QK, V, and RST operator pools; the two RW-to-address probes; the Q/K/V query projection; the RST query projection; and their `DirectTau` projections are shared across

all layers. Q and K share read/write rows but retain separate query slices, applicability thresholds, applicability values, and outputs. The two LayerNorms and attention output projection W_ℓ^O are layer-specific; positional and token embeddings and the final normalization are model-level parameters. Depth is not an explicit operator-pool index: it changes the representation state presented to the shared control interface.

C. Benchmark Metrics and Protocol Details

C4. All reported C4 rows use the same packed validation artifact and `packed_c4_validation_v1` protocol. The capped view contains 19,531 sequences. Exactly 19,520 sequences form 610 complete global evaluation batches, producing 9,974,720 valid next-token targets; 11 incomplete-tail sequences are excluded before scoring.

Zero-shot. The protocol is `mamba-table3_zero_shot_v1`, using stock `lm-eval==0.4.2`, global batch 32, maximum length 512, length buckets 64/128/256/512, and zero few-shot examples. Evaluated split sizes are 5,153 for LAMBADA, 10,042 for HellaSwag, 1,838 for PIQA, 2,376 for ARC-Easy, 1,172 for ARC-Challenge, and 1,267 for WinoGrande. Task, dataset, tokenizer, and software settings are identical between the two 400M models.

C.1. Detailed benchmark results

LAMBADA perplexities are 24.0774 ± 0.8005 for DAWN-SRW 400M, 24.0042 ± 0.8073 for dense 400M, and 17.6358 ± 0.5390 for DAWN-SRW 1.3B.

D. IOI Localization and Intervention Details

D.1. Frozen discovery

Discovery admits 119 paired-correct independent units and ranks 456,697 sites. Split-half rank stability is 0.913978765. The frozen selection rule is the smallest prefix satisfying cumulative absolute importance at least 0.70 and split top- k overlap at least 0.95. The selected $k = 4096$ prefix has:

- cumulative absolute importance 0.719248713;
- 3,993 shared split sites, overlap 0.974853516, and Jaccard 0.950940700;
- Q/K/V/RST counts = 1744/2350/2/0;
- coverage of every layer from 0 through 17.

Validation and test do not recompute or alter this selection.

Table 4. Model and training specification. The 1.3B row is descriptive and does not form a controlled scaling curve with the 400M rows.

Model	Parameters	Tokens	D	Layers	Heads	QK / V / RST rows	Checkpoint
DAWN-SRW 400M	393,800,708	39,999,504,384	2,304	18	36	4,600 / 14,500 / 29,300	76,293
Dense Transformer 400M	393,896,960	39,999,504,384	1,280	18	20	—	76,293
DAWN-SRW 1.3B	1,272,038,404	19,999,981,568	4,096	24	32	8,534 / 26,904 / 54,352	87,193

Table 5. Detailed settings from the registered training configurations and restored checkpoint manifests.

Setting	DAWN-SRW 400M	Dense 400M	DAWN-SRW 1.3B
Tokenizer vocabulary	30,522	30,522	30,522
Residual width	2,304	1,280	4,096
Layers	18	18	24
Heads / head width	36 / 64	20 / 64	32 / 128
Maximum sequence length	512	512	512
Feed-forward width	—	5,120	—
Control dimension d_c	256	—	256
QK / V / RST operator-pool size	4,600 / 14,500 / 29,300	—	8,534 / 26,904 / 54,352
QK / V / RST chunks	1 / 1 / 4	—	1 / 2 / 8
QK / V / RST normalizer exponent p_a	0.5 / 1.0 / 1.2	—	0.5 / 1.0 / 1.2
Initial applicability-support fraction targets	0.05 / 0.03 / 0.01	—	0.05 / 0.03 / 0.01
Training batch	1,024	1,024	448
Mesh (data $\times$ model)	16×2	16×2	16×2
Learning rate	3×10^{-4}	3×10^{-4}	2×10^{-4}
Warmup ratio	0.08	0.08	0.06
Weight decay	0.1	0.1	0.1
Applicability-threshold learning-rate multiplier	0.001	—	0.001
Dropout / <code>router.dropout</code>	0 / 0	0 / —	0 / 0
Gradient checkpointing	yes	yes	yes
Seed	1	1	1

Table 6. Packed C4 validation on the identical 9,974,720 target tokens.

Model	CE	PPL	Accuracy
DAWN-SRW 400M	2.940213	18.919873	0.424912
Dense 400M	2.935894	18.838336	0.427100
DAWN-SRW 1.3B	2.835879	17.045382	0.437099

D.2. Suppression, controls, and restoration

Let T_{corr} and T_{src} denote the correct-answer and source-answer token sets. The primary score is

$$\mathcal{M} = \sum_{t \in T_{\text{corr}}} \log p(t) - \sum_{t \in T_{\text{src}}} \log p(t).$$

Suppression zeros the frozen sites’ execution numerator at specified token positions. It leaves applicability and the normalization factor unchanged. Controls use the same 4,096-site count and exclude all circuit sites. Activation matching conditions on layer, computation type, and the discovery mean pre-scale operator-output-norm quantile. Restoration executes a distinct intervention: after suppression, it adds back the exact selected same-example numerator under the unchanged normalization factor.

On validation, restoration returns the margin to 3.462791, leaving residual drop 0.002665. On held-out test, it returns the margin to 3.242905, leaving residual drop 0.011815. The respective recovery fractions are 0.999661155 (95% CI [0.998654544, 1.000584125]) and 0.998405645 (95% CI [0.997323737, 0.999385266]).

E. Fail-Closed Results

RAVEL. Discovery uses 142 independent units. Rank stability is 0.628762 for Continent, 0.676243 for Country, and 0.660622 for Language, below the preregistered requirement. No prefix passes the complete discovery rule, no variable circuit is frozen, and validation, transfer, and test are not opened.

ARC. Discovery stability is 0.89386 and a 4,096-site Q/K-only circuit is frozen. On 51 validation units, intact margin is 1.735534 and suppressed margin is 2.819139, giving a frozen margin drop of -1.083605 (95% CI $[-1.472180, -0.690348]$). The effect has the wrong sign and fails control separation. Restoration recovery is 1.004083 (95% CI [0.994587, 1.013652]), but restoration alone does not pass the conjunction. Held-out test remains closed.

1.3B IOI. The scale replication admits 125 independent units and passes rank stability at 0.907187749. The largest allowed 4,096-site prefix reaches only 0.635144 cumulative absolute importance, below the 0.70 criterion. No circuit is frozen; validation and test remain closed.

F. Compute Accounting

The implementation shards operator rows across the model mesh and processes them in dense chunks. The 400M configuration uses one QK chunk, one V chunk, and four RST chunks. Every query scores all local operator addresses,

Table 7. Zero-shot accuracy. Parentheses contain standard errors. HS, AE, and AC denote HellaSwag, ARC-Easy, and ARC-Challenge.

Model	LAMBADA	HS	PIQA	AE	AC	WinoGrande	Mean
DAWN-SRW 400M	0.352028 (0.006654)	0.458375 (0.004972)	0.712187 (0.010563)	0.484007 (0.010255)	0.256826 (0.012767)	0.513812 (0.014047)	0.462872
Dense 400M	0.353192 (0.006659)	0.457080 (0.004971)	0.698041 (0.010712)	0.475589 (0.010248)	0.249147 (0.012639)	0.524862 (0.014035)	0.459652
DAWN-SRW 1.3B	0.395886 (0.006813)	0.510157 (0.004989)	0.723069 (0.010440)	0.509680 (0.010258)	0.268771 (0.012955)	0.507498 (0.014051)	0.485844

Table 8. Control effects and frozen-minus-control separation. Each control family contains 100 replicates. All paired permutation p -values are 0.0004997501 with 2,000 permutations.

Control	Validation drop	Frozen minus control (95% CI)	Test drop	Frozen minus control (95% CI)
Equal-count	0.070886	7.793902 [7.437851, 8.161572]	0.080276	7.330057 [6.922408, 7.716879]
Same-layer	0.039389	7.825399 [7.440629, 8.180288]	0.041073	7.369260 [6.977447, 7.754539]
Activation-matched	5.400490	2.464297 [2.076200, 2.832697]	5.472579	1.937754 [1.564818, 2.319706]
Route-frequency	0.219162	7.645626 [7.289512, 8.031625]	0.242665	7.167668 [6.779836, 7.545537]

Table 9. Fail-closed results. Later phases were not opened after a failed preregistered criterion.

Study	Passed	Failed criterion	Decision
RAVEL 400M	Behavioral eligibility	Variable stability: 0.628762 / 0.676243 / 0.660622	No frozen circuit; no validation, transfer, or test
ARC 400M	Stable discovery; 4,096 Q/K sites; restoration	Suppression effect wrong sign; control separation failed	Reject causal replication; held-out test closed
IOI 1.3B	Behavioral eligibility; rank stability 0.907188	4,096-site cumulative importance 0.635144 < 0.70	No frozen circuit; validation and test closed

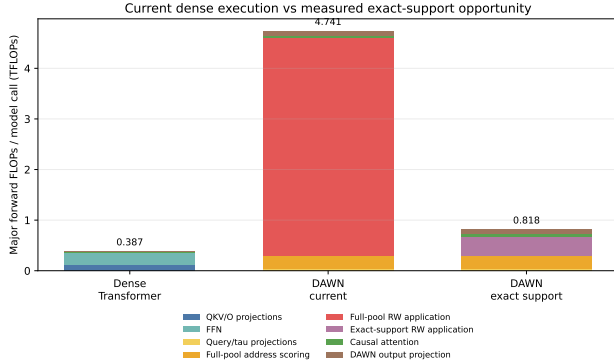


Figure 2. Static backbone accounting at batch 1 and sequence length 512. Address scoring remains dense in the estimate restricted to exact applicability support. Current dense RW execution also applies all full-dimensional RW rows. No production latency claim is made.

and every operator row participates in dense read and write contractions before cross-shard reduction. RW operators execute in the full residual dimension D ; the d_c -dimensional operation-address space is used only for applicability.

The applicability-support analysis evaluates 9,994,240 packed-C4 positions and observes applicability scores and applicability thresholds before RW application. It stores aggregate statistics, not per-token operator identities or vectors. Static accounting uses batch 1, sequence length 512, and 1 MAC = 2 FLOPs. It counts major matmul/einsum operations and omits softmax, normalization, applicability elementwise arithmetic, embedding lookup, residual addi-

tions, and communication collectives.

Static forward accounting is 4.741260116 TFLOPs for current dense DAWN-SRW, 0.818230250 TFLOPs for dense addressing plus RW application restricted to exact applicability support, and 0.386547057 TFLOPs for the matched dense control. The estimate restricted to exact applicability support is 17.2577% of current DAWN-SRW accounting but still 2.1168 times the dense Transformer. These are static arithmetic estimates, not measured latency, energy, or hardware speedup.

The estimate restricted to exact applicability support is mathematically output-equivalent if the executor can skip all and only zero-weight RW terms. It does not include indexed address retrieval and does not demonstrate kernel parity. Instrumented profiling time is excluded from latency claims because histogram collection is part of the measured path.

G. Reproducibility Identities

The experiments reported here use DAWN-SRW implementation revision v4.1.7.2, recorded under the repository model identity `spatial-rl-v4.1.7.2`; literal repository filenames and artifact paths retain the historical shorthand v4172.

The machine-readable checkpoint registry is `configs/paper_checkpoint_registry.yaml`. Exact evaluation commits, software versions, data hashes, result roots, and complete artifact hashes are recorded in the repository experiment ledger. Historical experiment branches are evidence identities only; this source is

Table 10. Backbone static accounting. “Exact applicability support” retains full dense address scoring and replaces only full-pool RW application with the measured applicability support according to $c_{a,i}(x) > 0$.

Component	Current dense DAWN-SRW	Dense address + exact RW	Dense Transformer
Address scoring + query/threshold projections	0.293742	0.293742	–
RW application	4.306187	0.383157	–
Causal attention	0.043487	0.043487	0.024159
Dense output / QKV / FFN maps	0.097845	0.097845	0.362388
Total TFLOPs	4.741260	0.818230	0.386547

Table 11. Primary immutable identities. Branch names are omitted because they move; exact checkpoint steps, hashes, and artifact locations are the binding identities.

Evidence	Exact identity	Repository source
DAWN-SRW 400M	model spatial-r1-v4.1.7.2; step 76,293; checkpoint identity a7ce8afc...f91c17; config hash 08733ae4...5039b	configs/train_config_v4172_400M_c4_40B_v4_64_ver1_den_qk0p5_vlp0_rst1p2.yaml
Dense 400M	model baseline-JAX; step 76,293; config hash c3ce5d72...82fc25	configs/train_config_baseline_tpu_400M_c4_40B_v4_64_tp2.yaml
DAWN-SRW 1.3B	model spatial-r1-v4.1.7.2; step 87,193; checkpoint identity 2e38b41f...bf5f9; config hash 1e877263...4a5222	configs/train_config_v4172_1p3B_c4_20B_v4_64_ver1_den_qk0p5_vlp0_rst1p2.yaml
IOI circuit	parsed frozen-spec hash 6ec26618...418c; site hash 573cfff8...0200; selected-row hash 1439f6f3...fe0c1	configs/paper_ioi_frozen_circuit_v4172_400m.yaml
Compute support	support hash bleac5f5...bdc2; accounting hash d7ad6115...984e	assets/paper/v4172_400m_compute_support_step76293/

maintained on the canonical main branch.

Acknowledgments

The author thanks Anthropic Claude, Claude Code, OpenAI ChatGPT, and OpenAI Codex for assistance throughout this research. This work was developed with extensive use of AI systems for literature exploration, conceptual exploration, cross-domain framing, abstraction and terminology refinement, implementation support, debugging, and manuscript preparation. In particular, these systems were used as interactive assistants for refining the learned state-transition formulation and the exposition of DAWN-SRW. The research direction, architectural decisions, experimental design, experiments, analyses, interpretations, and final claims remain the responsibility of the author.

This research was supported by the Google TPU Research Cloud (TRC) program, which provided computational resources for training and analysis.

References

- Berges, V.-P., Oğuz, B., Haziza, D., Yih, W.-t., Zettlemoyer, L., and Ghosh, G. Memory layers at scale. *arXiv preprint arXiv:2412.09764*, 2024. doi: 10.48550/arXiv.2412.09764.
- Chen, Y., Gu, N., Shang, J., Zhang, Z., Feng, Y., Sheng, J., Liu, T., Wang, S., Sun, Y., Wu, H., and Wang, H. Mixture of universal experts: Scaling virtual width via depth-width transformation. *arXiv preprint arXiv:2603.04971*, 2026. doi: 10.48550/arXiv.2603.04971.
- Cloutier, J. R. State-dependent riccati equation techniques: An overview. In *Proceedings of the 1997 American Control Conference*, volume 2, pp. 932–936, 1997. doi: 10.1109/ACC.1997.609663.
- Csordás, R., Irie, K., Schmidhuber, J., Potts, C., and Manning, C. D. MoEUT: Mixture-of-experts universal transformers. In *Advances in Neural Information Processing Systems*, volume 37, 2024a. doi: 10.52202/079017-0897.
- Csordás, R., Piekos, P., Irie, K., and Schmidhuber, J. SwitchHead: Accelerating transformers with mixture-of-experts attention. In *Advances in Neural Information Processing Systems*, volume 37, 2024b. doi: 10.52202/079017-2368.
- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, Ł. Universal transformers. In *International Conference on Learning Representations*, 2019.
- Didolkar, A., Goyal, A., Ke, N. R., Blundell, C., Beaudoin, P., Heess, N., Mozer, M., and Bengio, Y. Neural production systems. In *Advances in Neural Information Processing Systems*, volume 34, pp. 25673–25687, 2021.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Foerster, J. N., Gilmer, J., Sohl-Dickstein, J., Chorowski, J., and Sussillo, D. Input switched affine networks: An RNN architecture designed for interpretability. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1136–1145, 2017.
- Geva, M., Schuster, R., Berant, J., and Levy, O. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 5484–5495, 2021. doi: 10.18653/v1/2021.emnlp-main.446.
- Geva, M., Caciularu, A., Wang, K., and Goldberg, Y. Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 30–45, 2022. doi: 10.18653/v1/2022.emnlp-main.3.
- Ha, D., Dai, A., and Le, Q. V. Hypernetworks. In *International Conference on Learning Representations*, 2017.
- He, X. O. Mixture of a million experts. *arXiv preprint arXiv:2407.04153*, 2024. doi: 10.48550/arXiv.2407.04153.
- Huang, J., Wu, Z., Potts, C., Geva, M., and Geiger, A. RAVEL: Evaluating interpretability methods on disentangling language model representations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume*

- I: Long Papers*), pp. 8669–8687, 2024. doi: 10.18653/v1/2024.acl-long.470.
- Huang, M., Shi, H., Zheng, C., Wu, Y., Chen, G., Yu, X., Yin, Y., and Cheng, H. UniPool: A globally shared expert pool for mixture-of-experts. *arXiv preprint arXiv:2605.06665*, 2026. doi: 10.48550/arXiv.2605.06665.
- Kramár, J., Lieberum, T., Shah, R., and Nanda, N. AtP*: An efficient and scalable method for localizing LLM behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024. doi: 10.48550/arXiv.2403.00745.
- Lample, G., Sablayrolles, A., Ranzato, M., Denoyer, L., and Jégou, H. Large memory layers with product keys. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Lu, H., Zhao, C., Xue, M., Yao, L., Moore, K., and Gong, D. Little by little: Continual learning via incremental mixture of rank-1 associative memory experts. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. arXiv:2506.21035.
- Mittal, S., Raparthy, S. C., Rish, I., Bengio, Y., and Lajoie, G. Compositional attention: Disentangling search and retrieval. In *International Conference on Learning Representations*, 2022.
- Neelakantan, A., Le, Q. V., and Sutskever, I. Neural programmer: Inducing latent programs with gradient descent. In *International Conference on Learning Representations*, 2016.
- Plotkin, G. D. A structural approach to operational semantics. Technical Report DAIMI FN-19, Computer Science Department, Aarhus University, Aarhus, Denmark, September 1981.
- Rahaman, N., Gondal, M. W., Joshi, S., Gehler, P. V., Bengio, Y., Locatello, F., and Schölkopf, B. Dynamic inference with neural interpreters. In *Advances in Neural Information Processing Systems*, volume 34, pp. 10985–10998, 2021.
- Reed, S. and de Freitas, N. Neural programmer-interpreters. In *International Conference on Learning Representations*, 2016.
- Schlag, I., Irie, K., and Schmidhuber, J. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 9355–9366, 2021.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.
- Shen, Y., Zhang, Z., Cao, T., Tan, S., Chen, Z., and Gan, C. ModuleFormer: Modularity emerges from mixture-of-experts. *arXiv preprint arXiv:2306.04640*, 2023. doi: 10.48550/arXiv.2306.04640.
- Shi, J., Peng, Z., Zhu, Y., Wu, Y., Liu, G., and Luo, Y. OmniMoE: An efficient MoE by orchestrating atomic experts at scale. *arXiv preprint arXiv:2602.05711*, 2026. doi: 10.48550/arXiv.2602.05711.
- Sutskever, I., Martens, J., and Hinton, G. E. Generating text with recurrent neural networks. In *Proceedings of the 28th International Conference on Machine Learning*, pp. 1017–1024, 2011.
- Wang, K., Variengien, A., Conmy, A., Shlegeris, B., and Steinhardt, J. Interpretability in the wild: A circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations*, 2023.
- Weiss, G., Goldberg, Y., and Yahav, E. Thinking like transformers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 11080–11090, 2021.
- Weiss, M., Rahaman, N., Locatello, F., Pal, C., Bengio, Y., Schölkopf, B., Li, E. L., and Ballas, N. Neural attentive circuits. In *Advances in Neural Information Processing Systems*, volume 35, pp. 7741–7754, 2022. doi: 10.52202/068431-0562.
- Yang, Y., Wang, C., and Li, J. UMoE: Unifying attention and FFN with shared experts. In *Advances in Neural Information Processing Systems*, volume 38, 2025.